Note: This is a closed-book exam. You may not communicate with each other (or anyone else!) during the course of the exam.
For all code below, you must carry out the logic yourself. **Marks will be lost for poorly named, formatted and/or organised code (including poor package structure).**

- You may not use any Java API methods to duplicate the content of an array or display the content of an Array.
- You must use arrays for **all** operations. You may not use any of Java's Collection classes, i.e. no ArrayList etc.

Exam Context:

Your exam is designed around Student data. You have been provided a base project containing:

- Student.java– a class modelling a Student. Students contain:
  - String studentNum
  - String name
  - String courseName
  - int year
- StudentGenerator.java – a class providing generateStudent() – this will generate a new Student object

PART 1 : Pseudocode (minimum 15 minutes) [20 marks]:

These files should be placed in the src/main/resources/pseudocode folder of the supplied base project.

Scenario:

The college operates a student services office. Students join a **queue** on arrival & are seen in order of arrival.

Using a text editor (notepad++, wordpad, textpad or similar – no code editor or IDE use is permissible) write the pseudocode for the following actions in your queue implementation:

- Adding a student to the queue
- Removing and returning the next student in the queue
- Retrieve the next student in the queue (without removing it)
- Checking if the queue is empty

Your pseudocode must also state the information required to maintain the queue, e.g.:

- A storage structure such an array, or a linked structure based on nodes
- First and last references, or indexes within an array
- A counter to track the number of elements in the queue

Note: You can continue to work on the pseudocode after the initial 15 minutes of the exam if you wish. However, you may not work on implementation within your IDE until 15 minutes have elapsed.

PART 2: Data Structure Implementation (this cannot be started until 20 minutes into the exam):

Problem 1: Student queue [35 marks]

Implement a queue to manage students waiting for help. Your queue must provide the following actions:

1. enqueue(Student student) -> Adds a Student to the back of the queue
2. dequeue() -> Removes and returns the next Student in the queue
3. peek() -> Returns (but does not remove) the next Student in the queue
4. isEmpty() -> Returns if the queue is currently empty
5. size() -> Returns the number of Students currently in the queue

Requirements:

- You may choose an appropriate underlying data structure.
- You may not use Java Collections or existing queue implementations.
- Your implementation must maintain FIFO behaviour.
- You must handle attempts to remove or inspect an empty queue appropriately.

Note: You have been provided with unit tests to confirm if your implementations function correctly.

Problem 2: Student registration set [25 marks]

Implement a set to manage student numbers associated with a specific course. Each student number should be unique. Your set must provide the following actions:

1. add(String studentNum) -> Adds a student number to the set if it does not already exist
2. contains(String studentNum) -> Returns if a supplied student number exists in the set
3. remove(String studentNum) -> Removes a student number (if present)
4. size() -> Returns the number of unique student numbers

Requirements:

- You may choose an appropriate underlying data structure.
- You may not use Java Collections or existing queue implementations.
- Duplicate student numbers should not be stored

Note: You have been provided with unit tests to confirm if your implementations function correctly.

PART 3: Complexity analysis [20 marks]:

In a text file called "Complexity analysis.txt" (in your /src/main/resources folder), answer the following questions based on YOUR implementations:

1) What is the **worst-case time complexity** for each of your operations in your queue implementation? **Briefly** justify your answers. [8 marks]
2) What is the **worst-case time complexity** for each of the operations in your set implementation? **Briefly** justify your answers. [6 marks]
3) Consider an alternative implementation approach for your queue, using a different underlying data structure. Explain how this might impact the performance and space requirements of the queue. [6 marks]